Module 5: File Handling in Python

File Modes • Context Manager • Open / Create / Read • Write / Update / Delete • Hands-on

Introduction to Files

A file stores information on a computer so it can be accessed later—even after a program ends.

Text Files

- Store readable text and characters
- Often viewed in a text editor
- Eg : **Plain Text** : .txt, .csv,
  Source Code : .py, .json, .html
  Date : .json, .xml

Binary Files

- Store data as bytes in a specific format
- Usually need a suitable application to interpret
- Eg: **Executable** : .exe, .bin
  Images : .jpg, .png,
  Audio/video : .mp3, .mp4

Why do we use files?

- Keep data after a program finishes
- Store information such as expenses , contacts etc

How Basic File Operations Work

1	**File must be present**	To read an existing file, it must already be in the folder.
2	**Open the file**	Open it in the mode you need, such as read or write.
3	**Read the content**	Read the information stored in the file.
4	**Close the file**	Close it when you have finished using it.
5	**Write to a file**	Open in write mode and write text. Write mode can replace old content.
6	**Create a new file**	Use create mode when you want a new file that does not already exist.

Sample Basic program

Basic sequence: open → use → close

Basic file handling

```
file = open(‘notes.txt', 'r')
content = file.read()
print(content)
file.close()
```

What happens?

1. `open()` opens the file in read mode.
2. `read()` gets its contents.
3. `print()` displays the contents.
4. `close()` closes the file when finished.

Important

If you open a file manually, remember to close it after use. The file must exist for read mode (`r`).

Context Manager in Python

A context manager helps manage resources, such as files, by ensuring they are properly released after use.

Using `with`

```
with open('notes.txt', 'r') as file:
    content = file.read()
    print(content)

# Python closes the file automatically
```

Why use a context manager?

- **Automatic cleanup:** Closes a file automatically when the with block finishes.
- **Safer code:** Helps prevent files from being left open accidentally.
- **Simple syntax:** Uses the ‘with’ keyword.

Remember

Prefer `with open(...) as file:` in everyday file-handling code.

File Opening Modes

Choose a mode based on what you want to do

Mode	Meaning	File missing	Existing content	Comment
r	Read (default)	Error	Kept	Just look, don't touch.
w	Write	Created	Erased	Start with a blank page.
a	Append	Created	Kept	Add to the end, keep everything else.
x	Exclusive create	Created	Error if file exists	Create only if it's brand new.
r+	Read and write	Error	Kept	Edit what's already there.
w+	Write and read	Created	Erased	Start fresh, then use it.
a+	Append and read	Created	Kept	Add to the end, and read too.

File Operation: Open

Purpose, example, and key points

Purpose

Open a file before reading or writing it.

Example

```
file = open('notes.txt', 'r')
print(file.read())
file.close()
```

With Context Manager

```
with open('notes.txt', 'r') as file:
    print(file.read())
```

Key points

- `open()` returns a file object.
- The mode controls how the file is used.
- Read mode requires an existing file.
- Close the file when finished.

File Operation: Read

Purpose

Retrieve and display information stored in a file.

Example

```
file = open('notes.txt', 'r')
content = file.read()
print(content)

#Readline function
file.seek(0)
line = file.readline ()
print (line)

file.close()
```

Key points

- read() reads the remaining content.
- read(n) reads up to n characters
- readline() reads one line.
- The file must exist in `r` mode.

File Pointer, seek() and tell()

File Pointer

Indicates the current position in a file where the next read or write operation takes place. Keeps moving with operations

tell()

Returns the current file position.

Syntax: file.tell()

seek()

Moves the file pointer to a specified position.

Syntax: file.seek(position)

Example – Say 'notes.txt' has 'Hello World'

```
with open("notes.txt", "r") as file:
    print(file.tell())
    print(file.read(2))
    print(file.tell())
    file.seek(0)
    print(file.read(5))
```

Output: 0 | He | 5 | Hello

Remember

tell() checks the position. seek() changes the position. Reading moves the pointer forward.

File Operation: Write

Purpose, example, and key points

Purpose

Save text to a file; mode `w` replaces existing content.

Example of overwriting existing content

```
file = open('notes.txt', 'w')
file.write('Shopping list\n')
file.write('Milk\n')
file.close()
```

Example of appending to existing content

```
file = open('notes.txt', 'a')
file.write('Shopping list 2\n')
file.write('Bread\n')
file.close()
```

Key points

- `write()` writes a string.
- `\n` starts a new line.
- Mode `w` overwrites existing content.
- Use `a` to add content without replacing it.

File Operation: Create

Purpose

Create a new file that does not already exist.

Example

```
file = open('new_notes.txt', 'x')
file.write('A new file')
file.close()
```

Key points

- Mode `x` creates a new file only.
- Existing file → `FileExistsError`.
- Mode `w` also creates a missing file, but overwrites it if it exists.

File Operation: Update

Purpose

Change existing file content by reading and writing.

Example

```
file = open('notes.txt', 'r+')
old_content = file.read()
print seek(0)
file.write('Updated notes\n')
file.seek(0)
new_content = file.read()
print (new_content)
file.close()
```

Key points

- `r+` reads and writes; the file must exist.
- `seek(0)` moves the position to the beginning.
- Shorter replacement text can leave old characters behind.
- For whole-file replacement, w+ would be simpler

File Operation: Delete

Purpose, example, and key points

Purpose

Remove a file that is no longer needed.

Example

```
import os

if os.path.exists('old_notes.txt'):
    os.remove('old_notes.txt')
else:
    print('File not found')
```

Key points

- `os.remove()` deletes the specified file.
- Check the path before deleting.
- Deleting a file differs from clearing its contents.
- You need permission to delete the file.

Hands-on: File Operations in Python

Task: Personal shopping list

Create a program that saves a shopping list in `shopping.txt`, reads it back, adds one more item, and displays the updated content.

Requirements

1. Create `shopping.txt`.
2. Write three shopping items.
3. Read and display the items.
4. Append one new item.
5. Read and display the updated file.

Extension

- Use a context manager for each operation.
- Handle a missing file gracefully.
- Print a message after each step.
- Optional: delete the file after checking the path.